

Algorithmus: Idee der Lösungsschreibung

Programm: Formulierung des Algorithmus in einer Prog.-Sprache

Software: Programm + Dokumentation

Hardware: Rechner + Peripherie

Eigenschaften eines Algorithmus

- in endlichem Text beschreibbar
- effektiv (d.h. durch eine Maschine ausführbar)
- Determinismus: es liegt eindeutig fest, welche Elementaroperation als nächstes ausgeführt wird
- Determiniertheit: zu jeder Eingabe gibt es genau eine Ausgabe
- Terminierung: Alg. hält immer nach endlich vielen Schritten an.

Determinismus, Determiniertheit, Terminierung werden nicht immer verlangt.

Bsp: Indet. Alg ist trotzdem determiniert.

Es gibt also einen "ersten" Algorithmus.

Eigenschaften eines "guten" Algorithmus

- Allgemeinheit: Alg. sollte nicht nur ein einziges Problem lösen, sondern ganze Klasse von Problemen.
- Änderbarkeit: Alg. sollte leicht modifizierbar sein.
- Effizienz: Anzahl der Rechenschritte sollte möglichst gering sein.
- Robustheit: sollte sich bei unzulässigen Eingaben wohldefiniert verhalten.

Bsp für Syntax + Semantik:

Sprache aller natürlichen Zahlen außer 0.

Syntax: Jede Zahl ist Folge von Ziffern
(0, 1, ..., 9), wobei erste Ziffer $\neq 0$.

Semantik: • Wert einer Zahl aus mehreren Ziffern
ist der Wert der letzten Ziffer plus
der 10-fache Wert der Zahl links davon.
• Wert einer Ziffer ist die Ziffer selbst.

z.B. 367

ist syntaktisch korrekt.

$$\begin{aligned} & \text{wert}(367) \\ &= \underbrace{\text{wert}(7)}_7 + 10 \cdot \text{wert}(36) \\ &= 7 + 10 \cdot (\underbrace{\text{wert}(6)}_6 + 10 \cdot \underbrace{\text{wert}(3)}_3) \\ &= 7 + 10 \cdot (6 + 10 \cdot 3) \\ &= 367 \end{aligned}$$

Dagegen: 007 gehört nicht zur Sprache.

Wie legt man die Syntax einer Sprache genau (formal) fest?

Bsp: Alphabet

- lat. Alphabet (a, b, ..., z)
- ASCII-Code (128 Zeichen) American Standard Code of Information Interchange
- $A_1 = \{0, 1\}$
- $A_2 = \{ (,), +, -, *, /, a \}$

Wörter

$$A_1^* = \{ \varepsilon, 0, 1, 00, 01, 10, 1010011, \dots \}$$

$$A_2^* = \{ \varepsilon, (), ((a+a)+a), (-+, \dots) \}$$

Sprachen

$$L = \{ \varepsilon, 1, 10, 1010011, \dots \}$$

Binärzahlen
ohne
führende
0en

$$\text{EXPR} = \{ \varepsilon, (), ((a+a)+a), \dots \}$$

Menge aller
korrekt geklammerten
Ausdrücke

Hier: nur Grammatiken

Automaten in Vorlesung FSAP (2. Semester)

Bsp

Satz $\Rightarrow$ Subj Präd Obj

$\Rightarrow$ Subj jagt Obj

$\Rightarrow$ Art Attr Subst jagt Obj

$\Rightarrow$

$\Rightarrow$ der große bissige Hund jagt die kleine Katze

Satz $\Rightarrow$ ~~...~~ $\Rightarrow$ die Katze der Hund

o.h. $\Rightarrow$ ~~...~~ $\Rightarrow$ der Hund jagt die Maus

Satz $\Rightarrow$ ~~X~~ $\Rightarrow$ der Hund jagt die Maus

Satz $\Rightarrow$... $\Rightarrow$ das große Hund jagt die große Hund

Grammatik

$$G = (N, T, P, S)$$

- N = endl. Menge von Nichtterminalsymbolen

Bsp: Satz, Subjekt, Prädikat, Objekt

- kommen in den Wörtern der Sprache nicht vor
- werden durch wiederholte Anwendung von Produktionsregeln ersetzt.

- T = endl. Menge von Terminalsymbolen

mit $N \cap T = \emptyset$

Bsp: der, die, das, Hund, Katze, ...

- Zeichen, aus denen die Wörter der Sprache bestehen

- P = endl. Menge von Produktionsregeln $X \rightarrow Y$

mit $X \in V^* N V^*$

$Y \in V^*$

wobei: $V = N \cup T$

Regel $X \rightarrow Y$ bedeutet, dass man das Teilwort x durch das Teilwort y ersetzen kann.

durch das Teilwort γ ersetzen kann.

Bsp: Subj Präd Obj $\Rightarrow$ Subj sagt Obj

• $S \in N$ das Startsymbol

S ist ein spezielles Nichtterminal, aus dem alle Wörter der Sprache hergeleitet werden.

Bsp: Satz

Ableitung:

ist eine Relation " $\Rightarrow$ " auf V^* .

Für $u, v, \gamma \in V^*$

$x \in V^* N V^*$ gilt:

$u x v \Rightarrow u \gamma v$, falls $x \rightarrow \gamma \in P$

Sprache der Grammatik G :

$L(G) = \{ w \mid w \in T^*, S \Rightarrow \dots \Rightarrow w \}$,

d.h., alle Terminalwörter, die aus dem Startsymbol S mit Hilfe der Produktionsregeln P ableitbar sind.

Zwei Grammatiken sind äquivalent, wenn sie die gleiche Sprache erzeugen.

wenn sie die gleiche Sprache erzeugen.

Eine Sprache ist Kontextfrei, falls sie von einer kontextfreien Grammatik erzeugt wird.

Bsp - Grammatik

Grammatik G ist nicht Kontextfrei:

Was ist $L(G)$?

$$A \Rightarrow a \underline{B} b c \Rightarrow d c$$

$$\Downarrow a a \underline{B} b b c \Rightarrow a d b c$$

$$\Downarrow a^3 \underline{B} b^3 c \Rightarrow a^2 d b^2 c$$

$$\Downarrow a^4 \underline{B} b^4 c \dots$$

$$L(G) = \{ a^n d b^n c \mid n \in \mathbb{N} \}$$

Diese Sprache ist trotzdem Kontextfrei, denn es ex. eine Kontextfreie Grammatik G' die äquivalent zu G ist.
Idee: Bauen die Wörter von hinten auf:

$$A \rightarrow B c$$

$$B \rightarrow a B b$$

$$B \rightarrow d$$

} B muss alle Wörter der Art $a^n d b^n$ erzeugen

EBNF

• Aus $A \rightarrow \gamma$ wird $A = \gamma$

- Aus $A \rightarrow \gamma$ wird $A = \gamma$
- Aus $A \rightarrow \gamma_1, \dots, A \rightarrow \gamma_n$ wird

$$A = (\gamma_1 \mid \gamma_2 \mid \dots \mid \gamma_n)$$

Kurnotation für

$$(\gamma_1 \mid (\gamma_2 \mid \dots (\gamma_{n-1} \mid \gamma_n) \dots))$$

Klammern werden oft weggelassen.

- Aus $A \rightarrow xz$ und $A \rightarrow xyx$ wird

$$A = x[y]z$$

- Aus $A \rightarrow xA$ und $A \rightarrow \gamma$ wird

$$A = \{x\}^* \gamma$$

Bsp-Grammatik in EBNF

Satz = Subj Präd Obj

Subj = Art Attr Subst

Artikel = $[("der" \mid "die" \mid "das")]$

Attribut = $\{Adj\}$

Adj = $("kleine" \mid "bissige" \mid "große")$

Subst = $("Hund" \mid "Katze")$

Präd = "jagt"

... ..

1raa - 00

Obj = Art Attr Subst

Syntaxdiagramme

- für jedes Nichtterminal ein Syntaxdiagramm
- Die Pfeile geben an, welche Wörter aus dem Nichtterminal hergeleitet werden können.
- Syntaxdiagramme können auch rekursiv sein, d.h. ein Nichtterminal kann in seinem eigenen Syntaxdiagramm auftreten.